

Universal Architecture & Integration

Contract

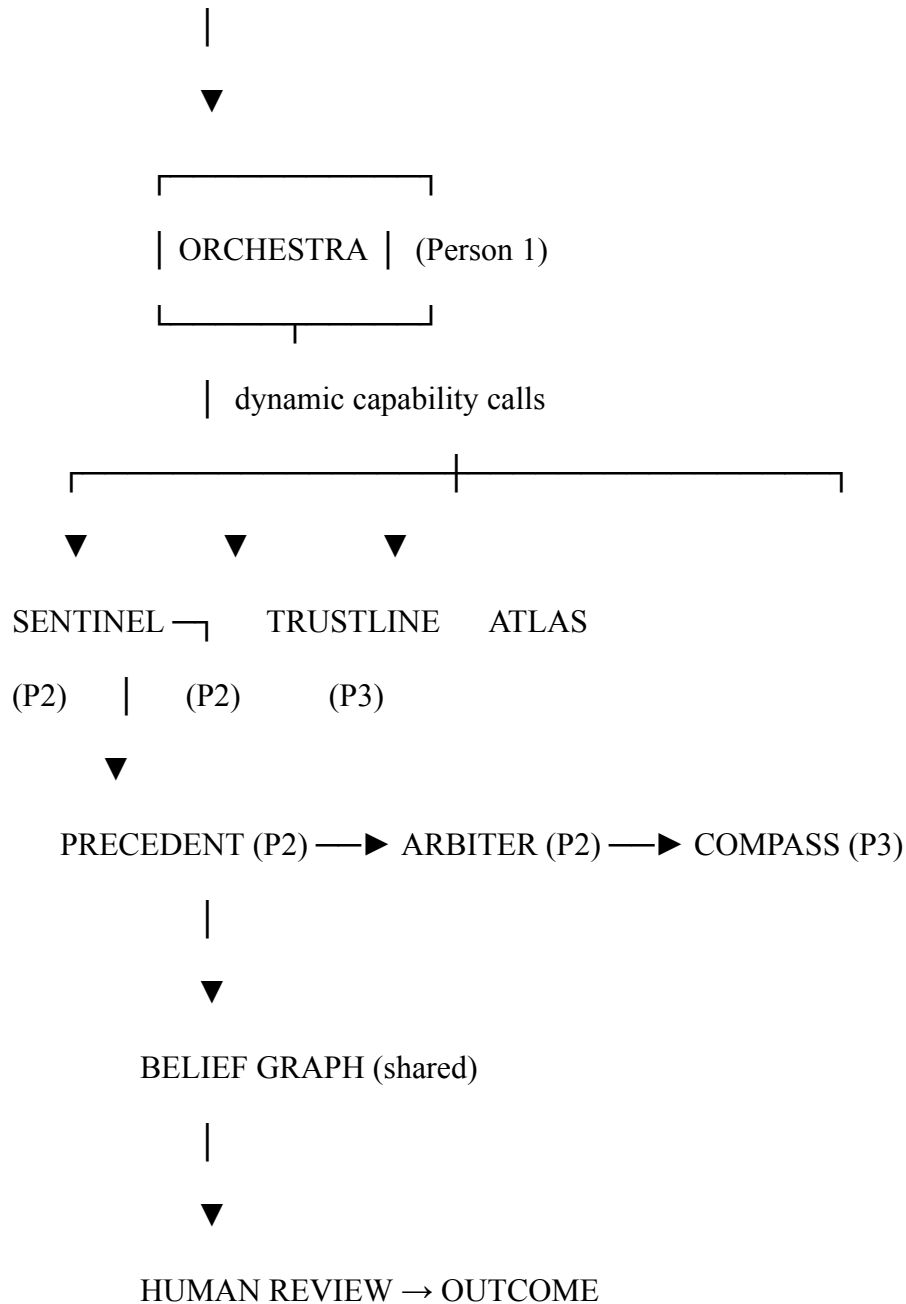
Procurement Intelligence System — ORCHESTRA + 5 Specialist Agents

This is the one doc everyone reads first and never deviates from. It defines the **shape** of the system so that Person 1, Person 2, and Person 3 can build in parallel and plug together without surprises. Role-specific implementation detail lives in the three companion docs ([01_PERSON1_ORCHESTRA.md](#), [02_PERSON2_INTELLIGENCE_EVIDENCE.md](#), [03_PERSON3_RISK_DECISION.md](#)).

1. The Core Idea (one paragraph)

ORCHESTRA is a dynamic AI orchestrator that looks at a procurement situation (a claim, a dispute, a delay, an invoice, an RFI), decides what it doesn't know, and calls **specialist capabilities** — never hardcoded workflows — until it has enough evidence to hand a human a decision. Six specialists (Sentinel, Trustline, Precedent, Arbiter, Atlas, Compass) each expose a narrow set of capabilities. Nobody calls anybody except through ORCHESTRA. Everything specialists learn gets written to a shared **Belief Graph/Ledger** so the system compounds knowledge over time instead of starting fresh every run.

PROCUREMENT DATA



Hard rule: specialists never call each other directly in code. They receive input from ORCHESTRA and return structured output to ORCHESTRA. This is what lets the *sequence* of calls change per situation without anyone's code changing.

2. Ownership (locked)

Owner	Systems
Person 1	ORCHESTRA Brain, Situation Understanding, Dynamic Planner, Capability Registry, Execution Graph, Evaluator, Replanning, Human-review routing
Person 2	Sentinel, Trustline, Precedent, Arbiter, RAG/Evidence infra
Person 3	Atlas, Compass
Shared	Belief Graph/Ledger, common schemas, evidence/receipt model, final integration, demo

3. Tech Stack (locked for the whole team — free/low-cost first)

Every choice below defaults to a free tier or fully free/local option so the team doesn't burn budget or hit rate limits mid-hackathon. Swap to paid tiers only if you actually hit a wall.

Layer	Choice	Why
Language	Python 3.11+ everywhere	one language, one venv, no context switching

API framework	FastAPI	every agent (Orchestra + 6 specialists) is a FastAPI service exposing its capabilities as endpoints
Orchestration engine	LangGraph (open source, free)	native support for conditional routing, parallel branches, loops, human-in-the-loop — matches the "no hardcoded workflow" design directly
Database + Vector store + File storage	Supabase (free tier: 500MB DB, 1GB file storage, pgvector extension available out of the box)	one hosted service gives you Postgres, vector search, document/photo storage buckets, and an auto-generated REST API — replaces separately standing up Postgres + Chroma + a file store
Realtime updates	Supabase Realtime (included free)	subscribe the demo frontend to task-graph/belief-ledger changes live — strong demo effect, no extra infra
Embeddings	Local sentence-transformers model (all-MiniLM-L6-v2), run as a tiny internal function/microservice	completely free, no API key, no rate limits, fast enough on CPU for a hackathon-scale corpus — avoids burning embedding-API quota on repeated re-ingestion during dev
LLM calls — primary	Google Gemini API (free tier) via common/llm_client.py	generous free-tier quota, large context window, good structured-output support — use for the bulk of situation understanding, verification reasoning, retrieval synthesis

LLM calls — high-volume/low-latency	Groq API (free tier, open models like Llama 3.1)	very fast + free — good for cheap classification-style calls (e.g. routing, quick relevance checks) where you don't need top-tier reasoning
Document text extraction	unstructured or pypdf (free, open source)	PDF/scan text extraction for evidence ingestion
Task/state passing between services	direct REST calls from ORCHESTRA only (each specialist is just an HTTP service Orchestra calls)	simplicity; no message broker needed at this scale
Auth between internal services	Supabase service-role key / a shared internal token — not user-facing auth	you're not building a public API
External risk data (Atlas)	free-tier sources where available (e.g. a free metals/commodity price API, a free exchange-rate API); for shipping/port congestion, use a curated realistic mock dataset unless you find a free live source in time	free live feeds for shipping/port data are scarce — don't burn build time chasing one; a well-labeled mock is fine for a demo
Containerization	Docker Compose for the FastAPI services only (Supabase is already hosted, so it's not part of your compose file — services just connect to it via URL + key)	anyone can docker compose up and have all agent services running locally against the same cloud Supabase project

Shared code	a <code>common/</code> Python package (schemas, evidence models, belief ledger client, <code>llm_client.py</code> , <code>supabase_client.py</code>) installed via <code>pip install -e ./common</code> into every service	prevents schema drift — this is the most important repo in the project
Frontend / demo UI	Nextjs + Supabase client (for realtime task-graph updates), or Streamlit if you want something faster to stand up	optional but strongly recommended for the demo; Streamlit is the safer bet if time is tight

Why Supabase specifically

- One free hosted Postgres instance with `pgvector` already available — no separate vector DB to run or pay for.
- **Storage buckets** give you a place to drop uploaded documents/photos (for Sentinel's evidence ingestion) without standing up S3 or local file handling.
- **Realtime** lets your frontend subscribe to the Belief Ledger / task graph table and show live updates as ORCHESTRA works through a situation — a good, low-effort demo moment.
- Auto-generated REST + row-level security if you end up needing quick read access from the frontend without writing extra endpoints.
- Everyone on the team hits the **same** database by default (one Supabase project, shared URL + keys) — removes a whole class of "works on my machine" integration bugs.

Repo structure (monorepo — recommended for a 3-person team on a deadline)

procurement-ai/

```

├── common/           # SHARED — build this together first
|
|   ├── schemas/     # Pydantic models: Task, Capability, Evidence, Receipt, Claim,
BeliefEdge
|
|   ├── belief_ledger/ # client for reading/writing the Belief Graph (via Supabase)
```

- | | — llm_client.py # wraps Gemini / Groq / Claude behind one interface
- | | — embedding_client.py # local sentence-transformers wrapper
- | | — supabase_client.py # single shared Supabase client (DB + storage + realtime)
- | — orchestra/ # Person 1
- | — sentinel/ # Person 2
- | — trustline/ # Person 2
- | — precedent/ # Person 2
- | — arbiter/ # Person 2
- | — atlas/ # Person 3
- | — compass/ # Person 3
- | — frontend/ # shared, low priority until core loop works
- | — docker-compose.yml # FastAPI services only — Supabase is external/hosted
- | — docs/ # these 4 docs live here

.env (shared, one Supabase project for the whole team)

SUPABASE_URL=...

SUPABASE_SERVICE_ROLE_KEY=...

GEMINI_API_KEY=...

GROQ_API_KEY=...

ANTHROPIC_API_KEY=... # optional, used selectively (Arbiter, Compass)

Put this in a **.env.example** in the repo root and have everyone create their own **.env** from it. Never commit real keys.

4. The Integration Contract (read this twice)

This is the piece that prevents Person 1 and Person 2/3 from drifting apart. **Freeze this before writing agent logic.**

4.1 Capability

Every specialist registers a fixed list of capabilities with ORCHESTRA's Capability Registry. A capability is just a name + input schema + output schema.

```
class Capability(BaseModel):
```

```
    name: str          # e.g. "sentinel.verify_claim"

    description: str    # used by the LLM planner to decide when to call it

    input_schema: dict  # JSON schema

    output_schema: dict # JSON schema

    endpoint: str       # e.g. "http://sentinel:8000/verify_claim"
```

4.2 Task (what ORCHESTRA sends a specialist)

```
class AgentTask(BaseModel):
```

```
    task_id: str

    capability: str      # "sentinel.verify_claim"

    project_id: str

    entity_ids: list[str] # vendor_id, contract_id, claim_id, etc.

    payload: dict        # capability-specific input

    context: dict | None = None # relevant Belief Graph snippet, prior findings
```

4.3 Agent Output (what every specialist returns — no exceptions)

```

class AgentResult(BaseModel):

    agent: str          # "sentinel"

    task_id: str

    status: Literal["completed", "needs_more_evidence", "failed"]

    findings: list[dict]

    claims: list[dict]

    evidence: list[str]    # evidence/receipt IDs, not raw blobs

    confidence: float      # 0-1

    risks: list[str] = []

    recommended_next_capabilities: list[str] = []

    receipt_id: str

```

Rule: no specialist ever returns a raw paragraph as its "real" output. Free text is allowed only inside a `findings[].explanation` field for human readability — the machine-usable fields above are mandatory.

4.4 Evidence & Receipt

```

class Evidence(BaseModel):

    evidence_id: str

    source_type: str      # "document", "photo", "report", "api"

    source_ref: str       # Supabase Storage path or URL / doc id

    extracted_text: str | None

    project_id: str

    linked_claim_ids: list[str]

```

```
class Receipt(BaseModel):

    receipt_id: str

    agent: str

    task_id: str

    inputs_hash: str          # for reproducibility/audit

    output_summary: str

    created_at: datetime
```

Uploaded documents/photos go into a Supabase Storage bucket (e.g. [evidence-files](#)); [Evidence.source_ref](#) stores the bucket path, not the raw file.

4.5 Belief Ledger edge (the shared knowledge graph)

```
class BeliefEdge(BaseModel):

    entity_id: str            # e.g. claim_id or vendor_id

    relation: str             # "supported_by", "contradicted_by", "affects", "caused_by"

    target_id: str

    confidence: float

    source_agent: str

    updated_at: datetime
```

Stored as a plain table in Supabase ([belief_edges](#)). Every specialist writes to it through one shared client ([common/belief_ledger/client.py](#)) built on [supabase_client.py](#) — nobody writes raw SQL against it directly.

5. Non-negotiable Design Rules

1. **ORCHESTRA is the only coordinator.** Specialists are stateless-per-call services; they don't know who called before them or who's called after.
 2. **All specialist outputs are structured (Pydantic), not prose.** Prose lives only in explanation fields.
 3. **No hardcoded "if claim then always call Sentinel then Trustline."** That sequencing decision belongs to ORCHESTRA's planner, driven by the Capability Registry descriptions.
 4. **One shared Supabase project, one shared schema package.** Five separate databases / five separate embedding stores is the #1 way this project fails to integrate in the final 24 hours.
 5. **Every agent action produces a receipt.** This is what makes the system auditable and is a strong demo point ("here's exactly why the AI flagged this vendor").
 6. **Human review is a first-class stop, not an afterthought.** Any decision above a risk/impact threshold routes to a human-approve/reject step before it's treated as final.
 7. **Route LLM calls by stakes, not by default.** Cheap/fast models (Gemini, Groq) for routine reasoning and classification; reserve Claude calls for the handful of steps where output quality is most visible (Arbiter, Compass) — keeps the whole team inside free-tier limits during development.
-

6. Build Order (team-wide, phased)

Do **not** let each person build all their systems fully before integrating. Integrate early and often.

Phase	Goal	Who
0	Freeze common/ schemas + stand up the shared Supabase project (tables, storage bucket, keys distributed)	Everyone, together, session 1
1	ORCHESTRA skeleton that can call ONE dummy capability end-to-end (even a stub)	Person 1

1	Sentinel: one real capability (verify_claim) wired to Supabase (DB + storage) and a basic local-embedding RAG pipeline	Person 2
1	Compass stub that takes any input and returns a fake recommendation (so Orchestra has an end-to-end path)	Person 3
2	Full vertical slice works: situation → Orchestra → Sentinel.verify_claim → Belief Ledger → Compass stub → human review	Everyone
3	Add Trustline, Atlas for real	Person 2 / 3
4	Add Precedent, Arbiter	Person 2
5	Add Compass real decision logic, human-review UI polish, replanning/loops in Orchestra	Person 1 / 3
6	Demo script: pick 2-3 scenarios that show dynamic re-routing (different capability sequences for different situations)	Everyone

Milestone that matters most: getting Phase 2's vertical slice working. That one thin end-to-end path de-risks integration more than any amount of individual feature-building.

7. What "done" looks like for the demo

Pick 2 scenarios where the **same system**, given different situations, calls a different sequence of capabilities — this is the actual proof that the dynamic orchestration idea works, not just a scripted pipeline. E.g.:

- Scenario A: vendor claims 80% manufacturing complete → `sentinel.verify_claim` → `sentinel.find_missing_evidence` → `trustline.get_vendor_profile` → `precedent.find_similar_case` → `arbiter.analyze_causation` → `compass.recommend`
- Scenario B: shipment delayed by port congestion → `atlas.assess_external_event` → `sentinel.verify_claim` → `trustline.update_trust` (no penalty, external cause) → `compass.recommend`

If Orchestra visibly takes two different paths through the same six specialists, that's the whole pitch. Bonus demo point: use Supabase Realtime to show the Belief Ledger updating live on screen as each capability runs.